

UNIT-II

Transport Layer

Topics to be discussed

1.The Transport Service

- a. **Services Provided to the Upper Layers,**
- b. **Transport Service Primitives,**

2.Berkeley Sockets

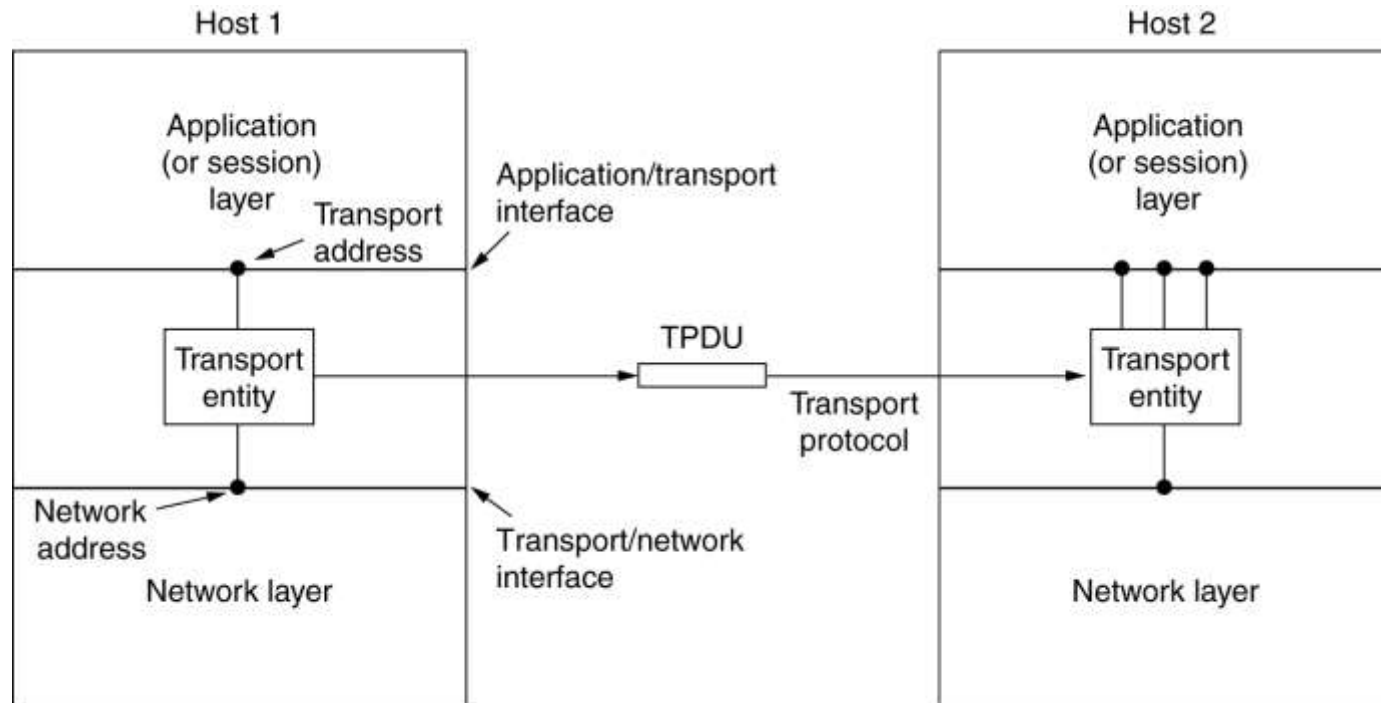
3.Elements of Transport Protocols,

- a. **Addressing**
- b. **Connection Establishment**
- c. **Connection Release**
- d. **Flow Control and Buffering**
- e. **Multiplexing**
- f. **Crash Recovery**

The Transport Service

Services Provided to the Upper Layers

The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective service to its users. hardware and/or software within the transport layer that does the work is called the **transport entity**.



The (logical) relationship of the network, transport, and application layers

Services Provided to the Upper Layers

2 types of services : (similar to network layer)

- **connection-oriented transport service :**
 - Has 3 phases : establishment, data transfer, and release.
 - Addressing and flow control are also similar network layer
- **connectionless transport service** is also very similar to the connectionless network service.

Why transport layer ?

- The users have no real control over the network layer, so they cannot solve the problem of poor service by using better routers or putting more error handling in the data link layer.

Services Provided to the Upper Layers

Difference ?

- The transport code runs entirely on the users' machines, but the network layer mostly runs on the routers, which are operated by the carrier (at least for a wide area network).
- Application programmers can write code according to a standard set of primitives and have these programs work on a wide variety of networks, without having to worry about dealing with different subnet interfaces and unreliable transmission.

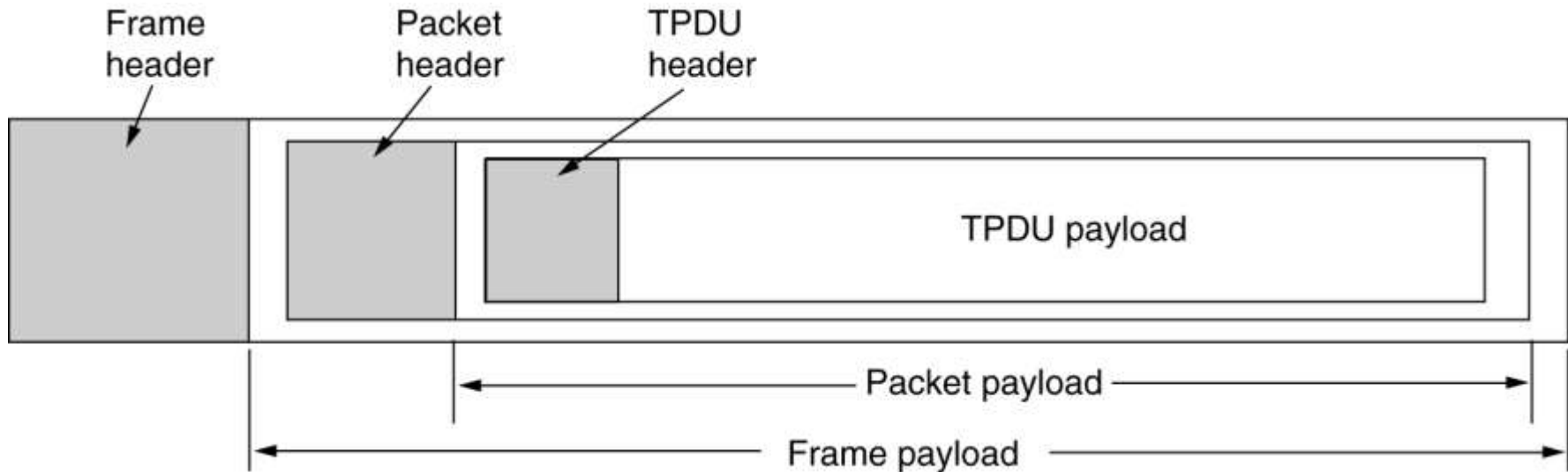
Transport Service Primitives

To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface. Each transport service has its own interface.

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

Transport Service Primitives

To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface. Each transport service has its own interface.



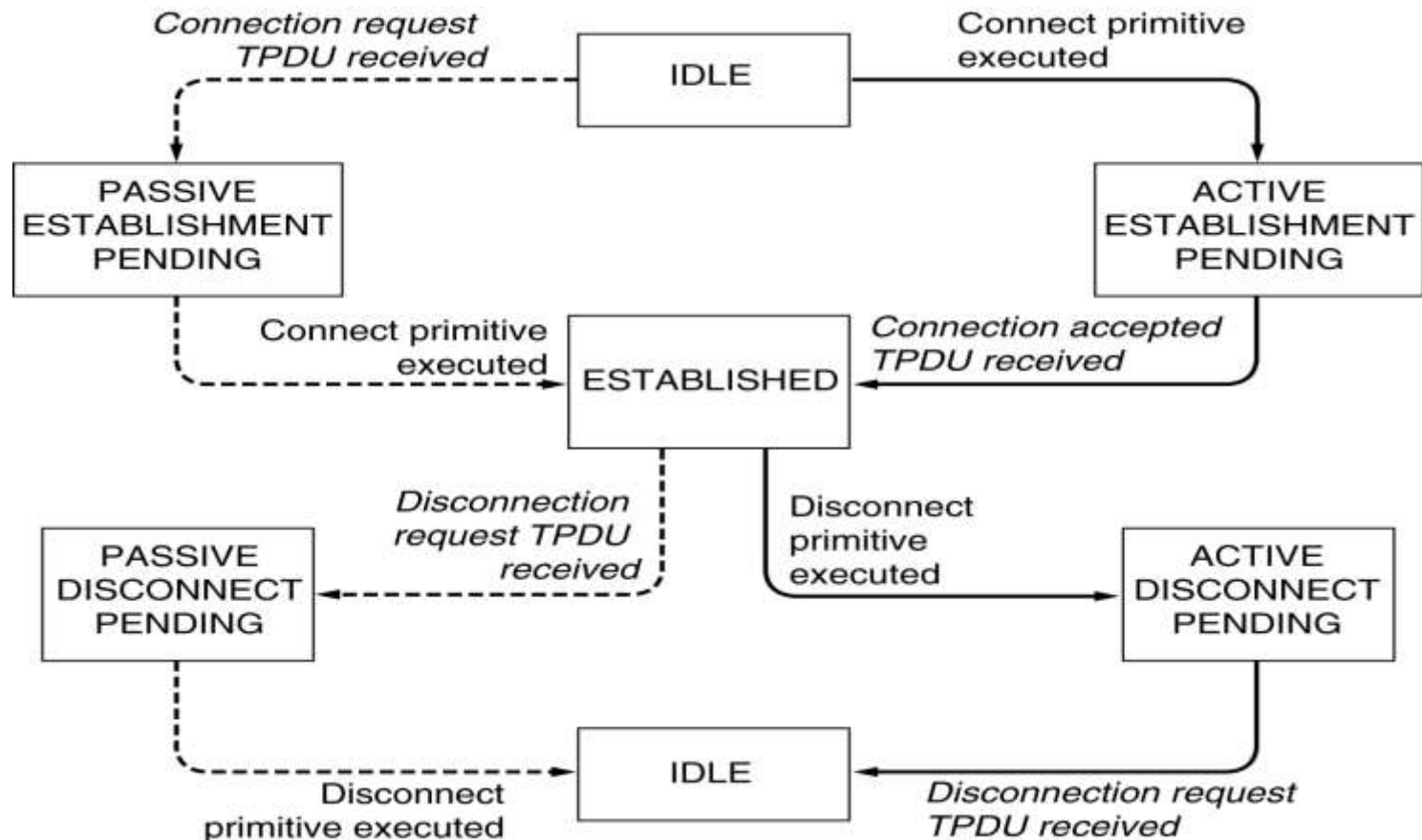
Transport Service Primitives

✓ A state diagram for a simple connection management scheme.

✓ Transitions labeled in **italics** are caused by packet arrivals.

✓ The **solid lines** show the client's state sequence.

✓ The dashed lines show the **server's state sequence**



Not in syllabus

Berkeley Sockets

set of transport primitives, the socket primitives used in Berkeley UNIX for TCP.

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

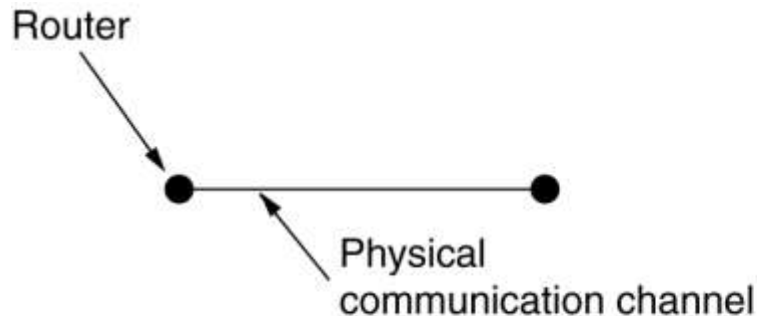
Elements of Transport Protocols

At the data link layer, two routers communicate directly via a physical channel, whereas at the transport layer, this physical channel is replaced by the entire subnet

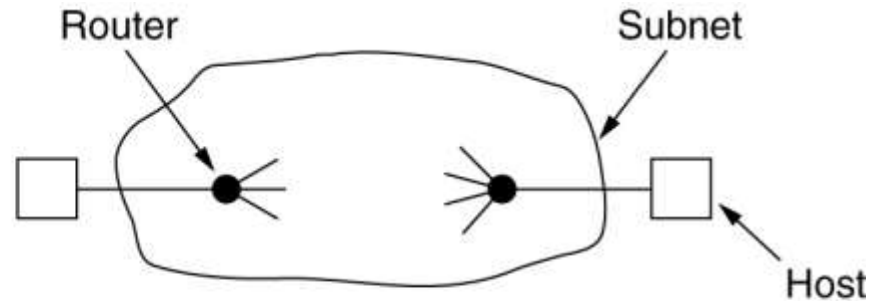
- (a) Environment of the data link layer.
- (b) Environment of the transport layer.

data link protocol v/s transport layer

At the data link layer, two routers communicate directly via a physical channel, whereas at the transport layer, this physical channel is replaced by the entire subnet



(a)



(b)

(a) Environment of the data link layer.

(b) Environment of the transport layer.

data link protocol v/s transport layer

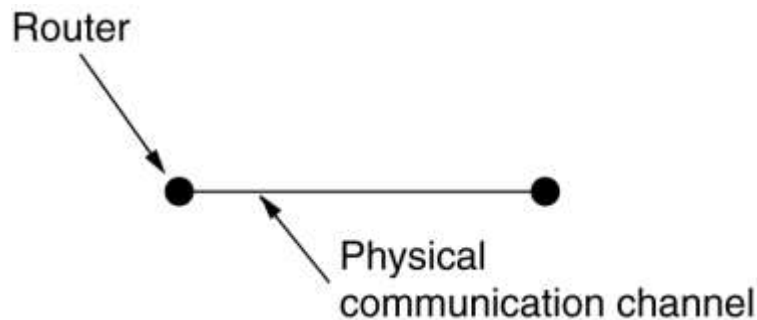
In the data link layer, it is not necessary for a router to specify which router it wants to talk to—each outgoing line uniquely specifies a particular router.

In the transport layer, explicit addressing of destinations is required.

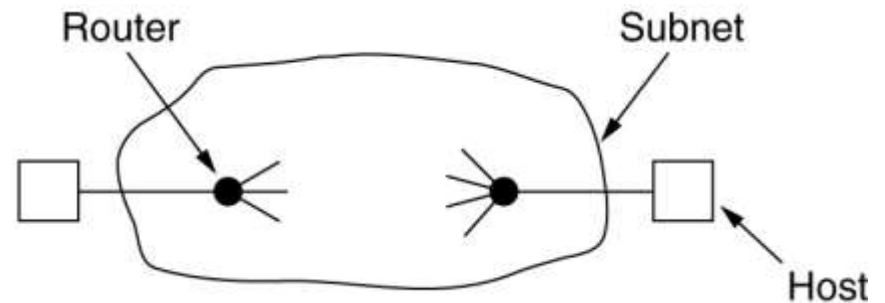
data link protocol v/s transport layer

In data link layer, the process of establishing a connection over the wire of is simple.

In the transport layer, initial connection establishment is more complicated.



(a)



(b)

data link protocol v/s transport layer

Potential existence of storage capacity in the subnet

- The consequences of the subnet's ability to store packets can sometimes be disastrous(disaster) and can require the use of special protocols.

data link protocol v/s transport layer

Presence of a large and dynamically varying number of connections in the transport layer

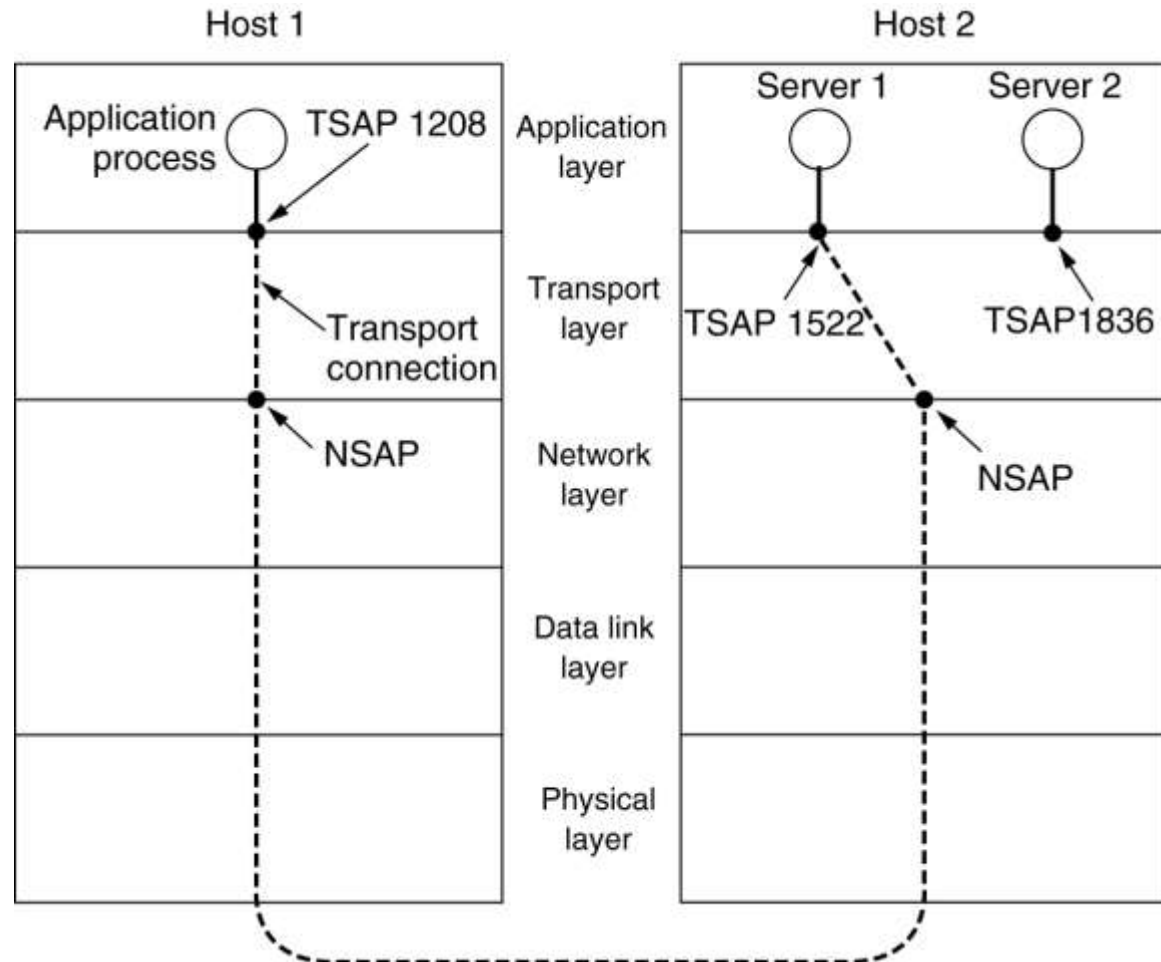
- The larger number of connections that must be managed make the idea of dedicating many buffers to each one less attractive and may require a different approach

Addressing

The method normally used is to define **transport addresses**

These end points are ***Transport Service Access Point***

TSAPs, NSAPs
and transport
connections.



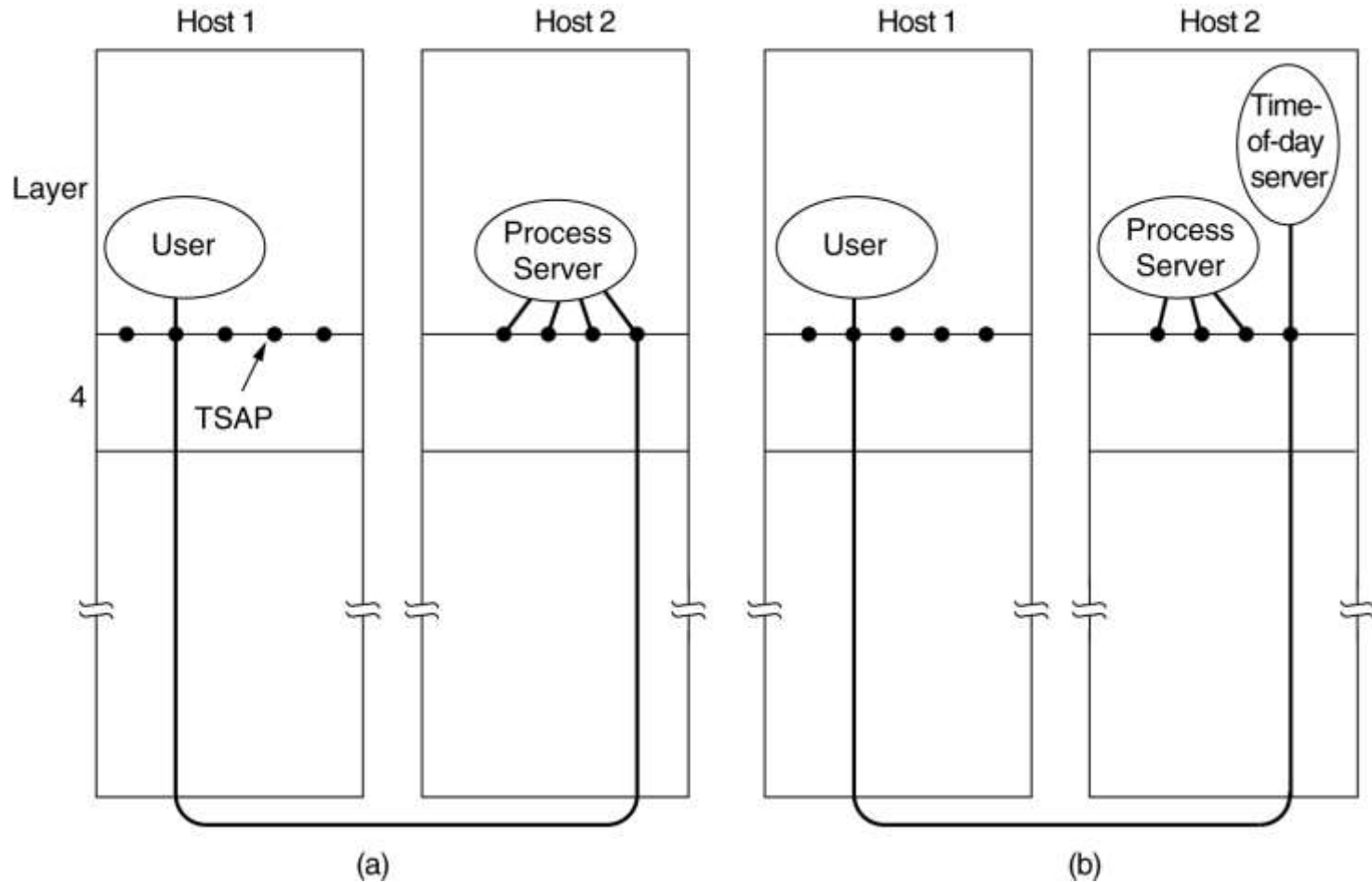
Addressing

How does the user process on host 1 know that the time-of-day server is attached to TSAP 1522?

- One scheme is known as the initial ***connection protocol***
- Each machine that wishes to offer services to remote users has a special process server that acts as a proxy

Addressing

Connection protocol



How a user process in host 1 establishes a connection with server1 in host 2.

Addressing

Connection using Name server or a directory server

- To find the TSAP address corresponding to a given service name, a user sets up a connection to the name server (which listens to a well-known TSAP).
- The user then sends a message specifying the service name, and the name server sends back the TSAP address.

Connection Establishment

Problem of existence of delayed duplicates

Solutions :

1. throw-away transport addresses

- In this approach, each time a transport address is needed, a new one is generated. When a connection is released, the address is discarded and never used again

Connection Establishment

Problem of existence of delayed duplicates

Solutions :

2. Give each connection a connection identifier

- After each connection is released, each transport entity could update a table listing obsolete connections
- It requires each transport entity to maintain a certain amount of history information indefinitely

Connection Establishment

Problem of existence of delayed duplicates

Solutions :

3. Ensure that no packet lives longer than some known time

1. Restricted subnet design.
2. Putting a hop counter in each packet.
3. Timestamping each packet

Connection Establishment

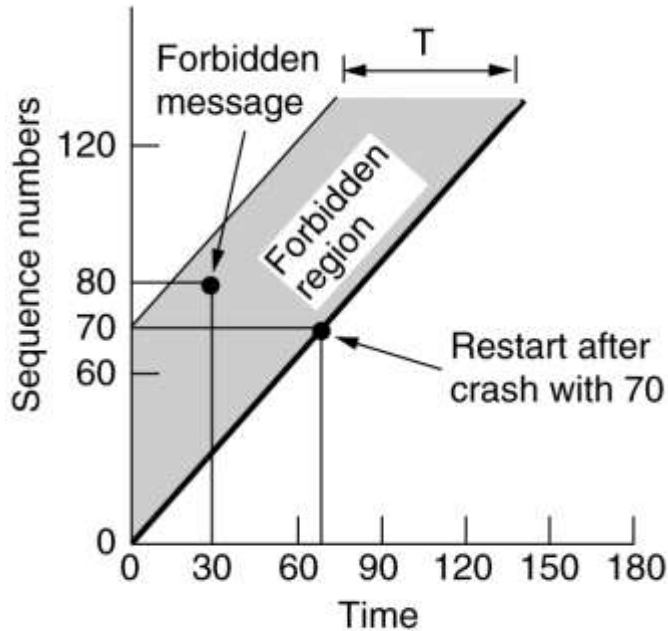
Problem of existence of delayed duplicates

Solutions :

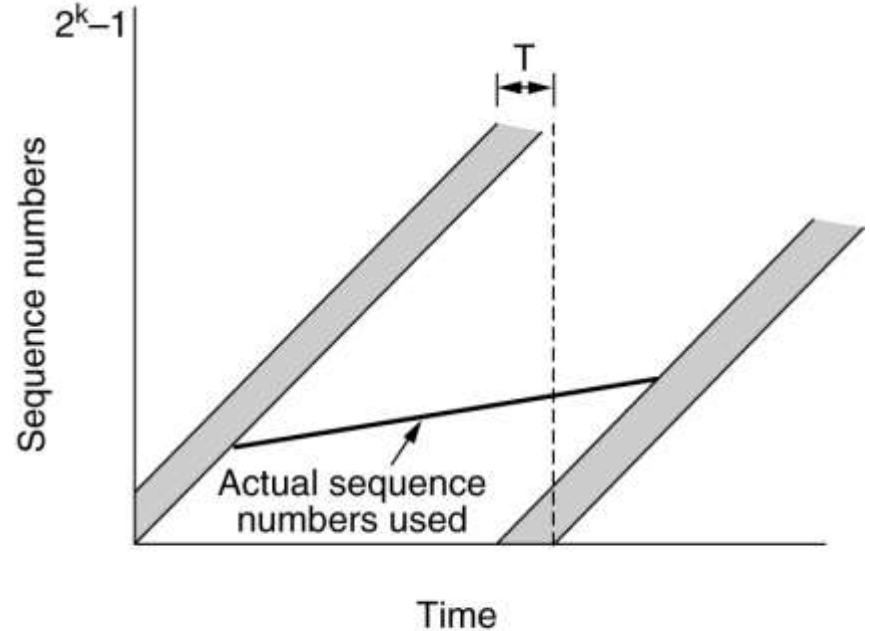
3. Ensure that no packet lives longer than some known time

1. Restricted subnet design.
2. Putting a hop counter in each packet.
3. Timestamping each packet

Connection Establishment



(a)

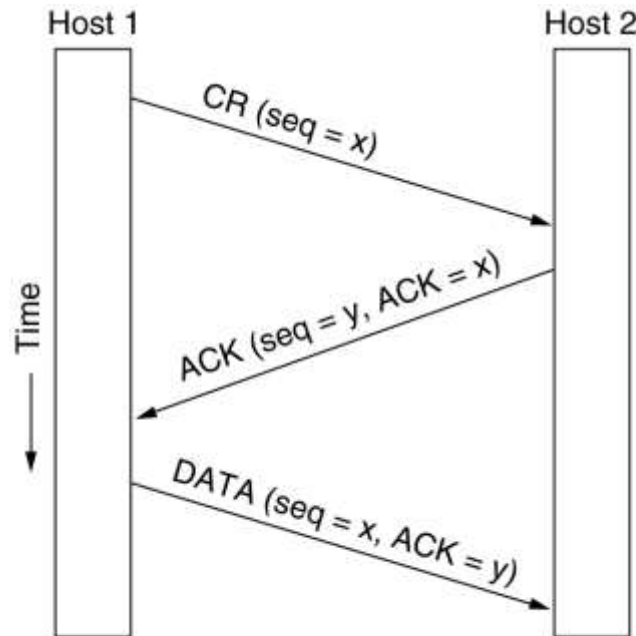


(b)

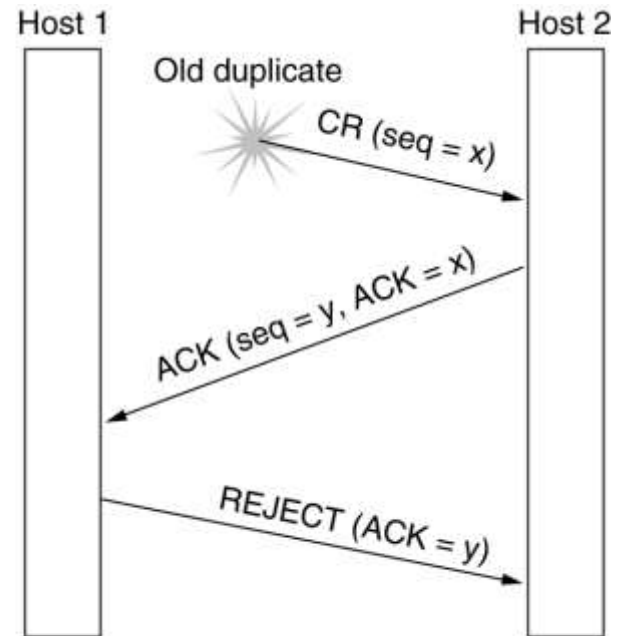
(a) TPDUs may not enter the forbidden region.

(b) The resynchronization problem.

3 three-way handshake. CR denotes CONNECTION REQUEST.



(a)



(b)

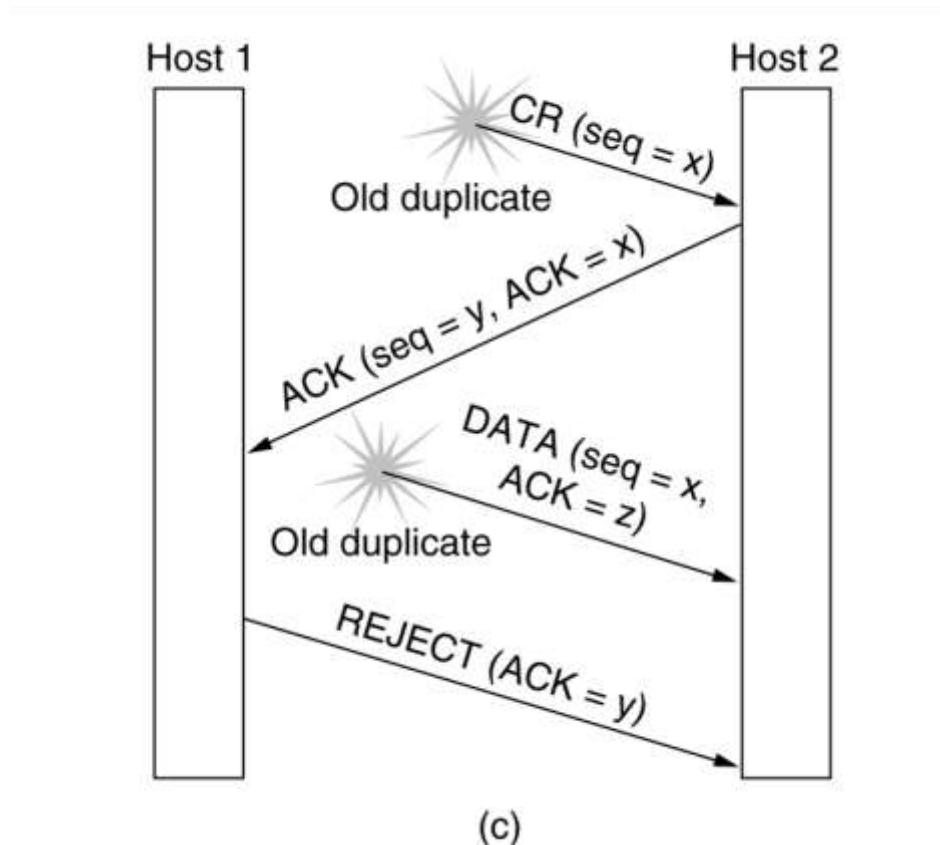
(a) Normal operation,

(b) Old CONNECTION REQUEST appearing out of nowhere.

Connection Establishment

3 three-way handshake. CR denotes CONNECTION REQUEST.

Duplicate CONNECTION REQUEST and duplicate ACK.



Connection Release

two styles of terminating a connection:

1.asymmetric release

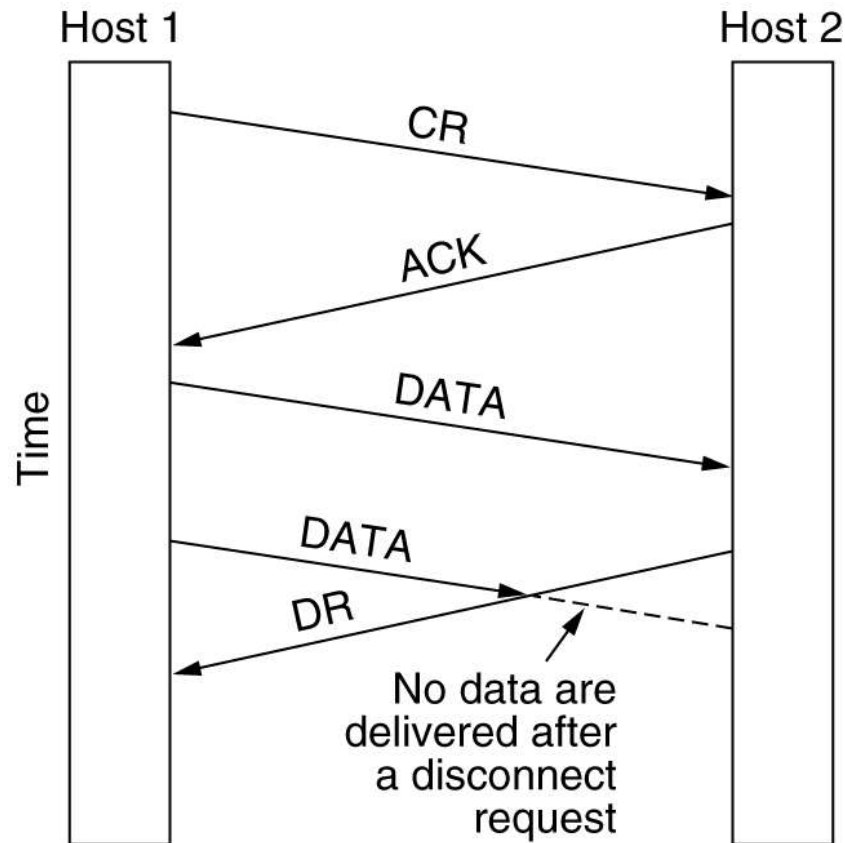
- Release of one unidirectional connection

2.symmetric release

- the connection as two separate unidirectional connections and requires each one to be released separately.

Connection Release

Asymmetric release is abrupt and may result in data loss.:



Abrupt disconnection with loss of data.

Connection Release

Symmetric release works like this :

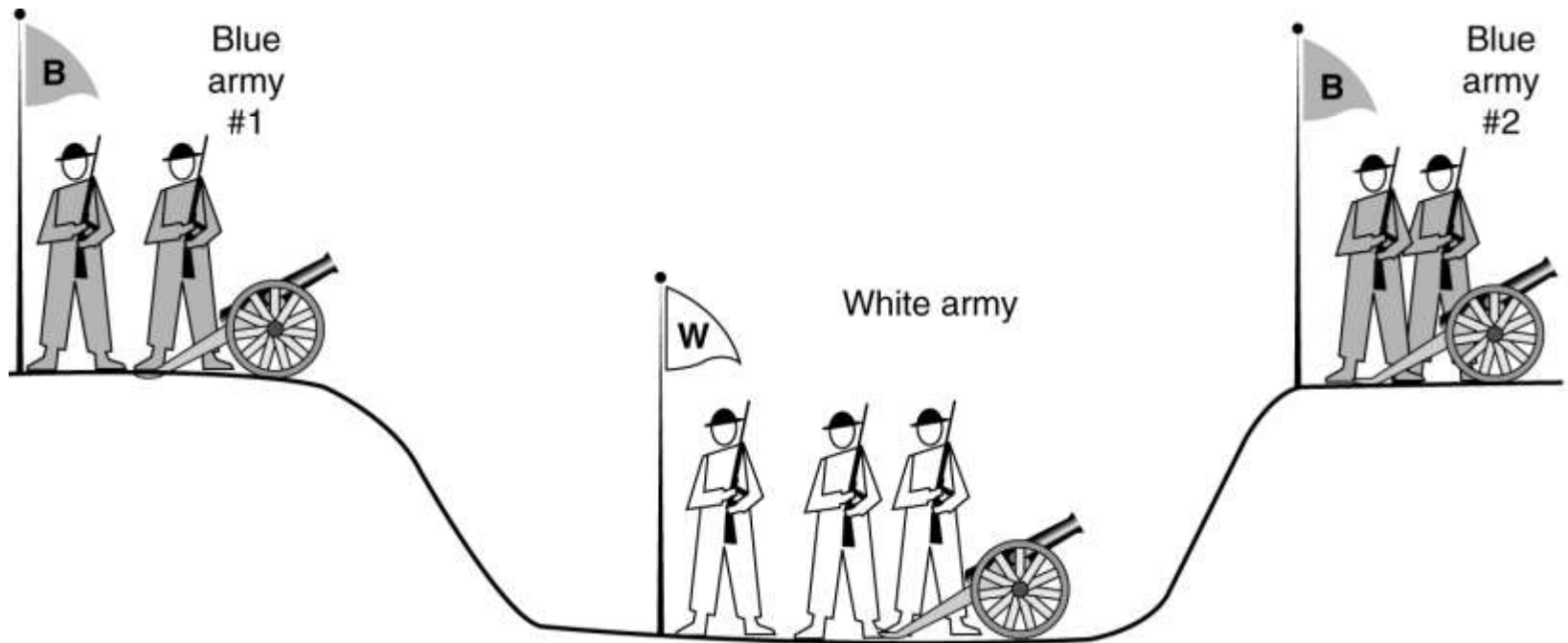
Host 1 says: I am done. Are you done too?

If host 2 responds: I am done too. Goodbye.

the connection can be safely
released. So no data loss

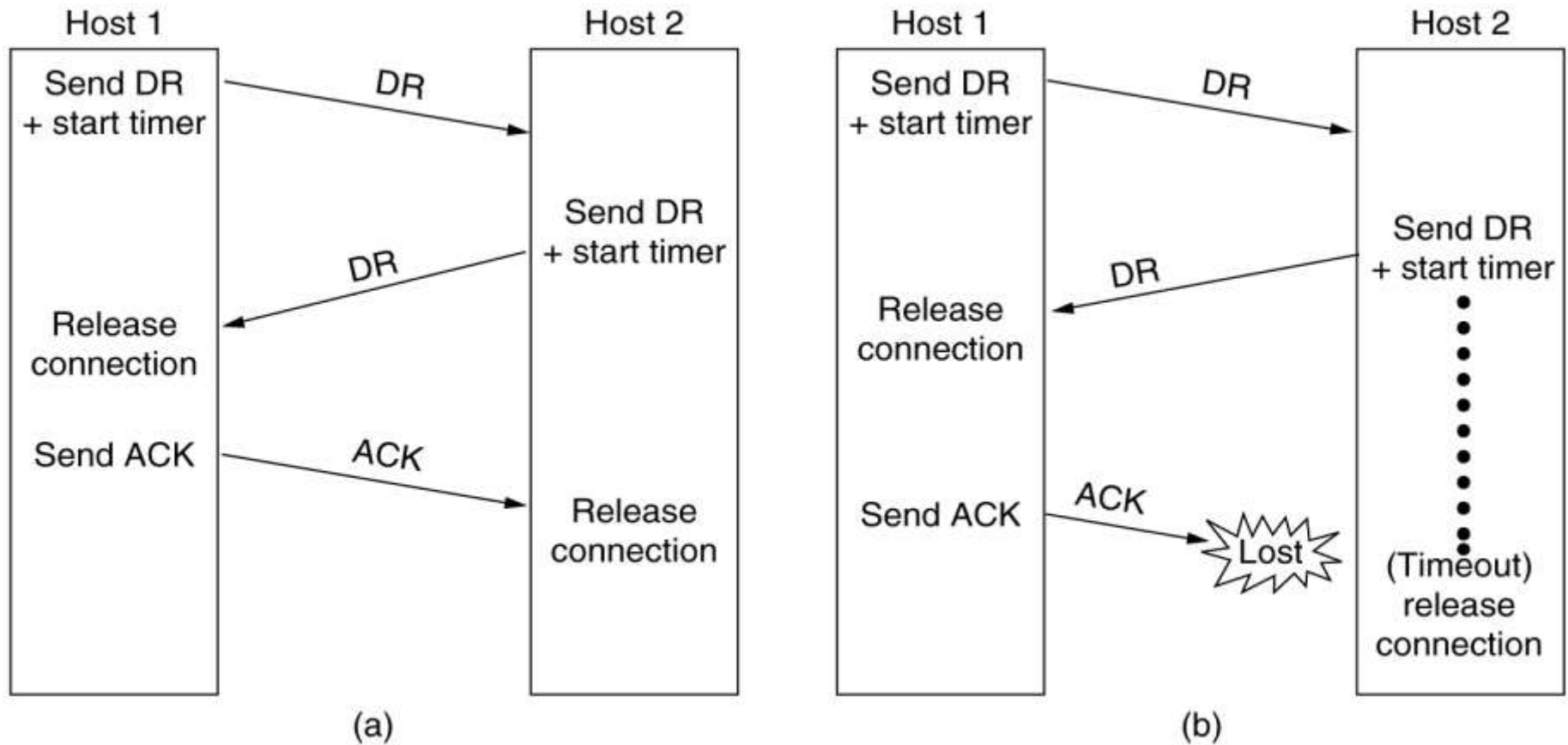
Connection Release

Two-army problem :



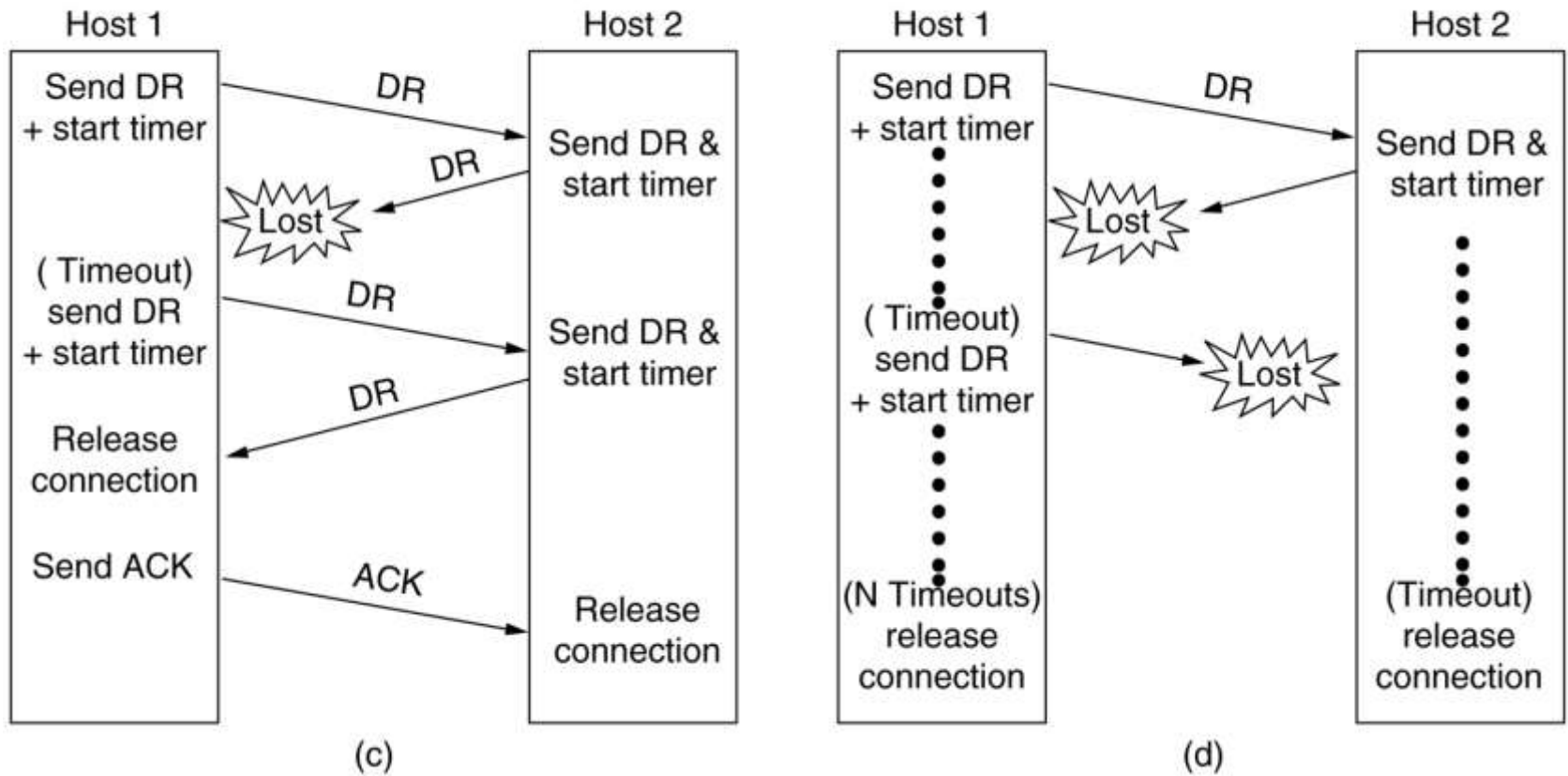
If neither side is prepared to disconnect until it is convinced that the other side is prepared to disconnect too, the disconnection will never happen.

Connection Release



Four protocol scenarios for releasing a connection. (a) Normal case of a three-way handshake. (b) final ACK lost.

Connection Release



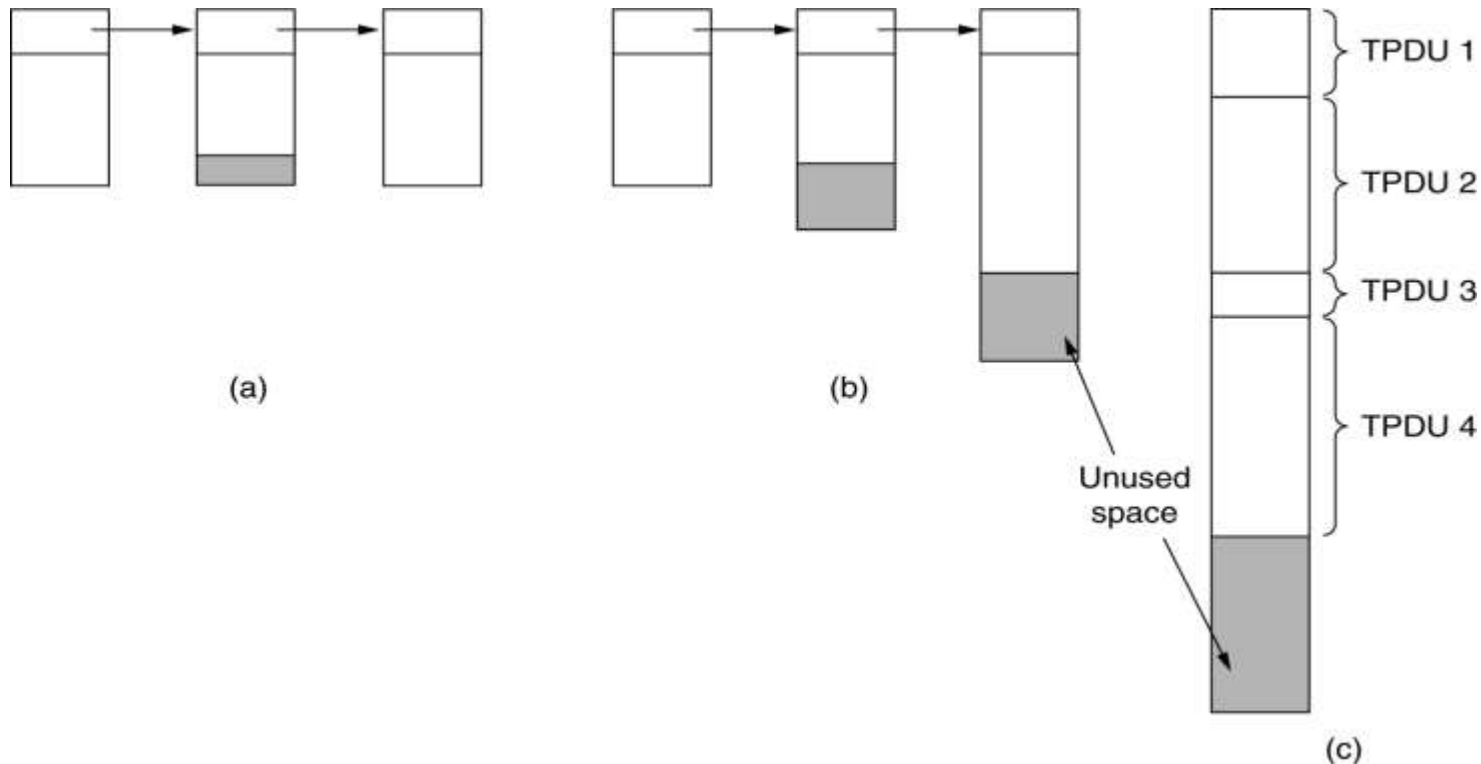
(c) Response lost. (d) Response lost and subsequent DRs lost.

Flow Control and Buffering

- It is impractical to implement the data link buffering strategy in the transport layer
- if the network service is unreliable, the sender must buffer all TPDU's sent
- with reliable network service
 - Sender need not retain copies of the TPDU's if receiver always has buffer space .
 - If receiver cannot guarantee that every incoming TPDU will be accepted, the sender will have to buffer anyway

Flow Control and Buffering

If the receiver has agreed to do the buffering, what about the buffer size?



- (a) Chained fixed-size buffers. (b) Chained variable-sized buffers.
(c) One large circular buffer per connection.

Flow Control and Buffering

The optimum trade-off between source buffering and destination buffering depends on the type of traffic carried by the connection.

- For low-bandwidth bursty traffic ☐ buffer at the sender
- For high-bandwidth traffic ☐ buffer at the receiver

Flow Control and Buffering

Managing dynamic buffer allocation

1. The sender requests a certain number of buffers,
2. The receiver then grants as many of these as it can afford
3. Every time the sender transmits a TPDU, it must decrement its allocation, stopping altogether when the allocation reaches zero
4. The receiver then separately piggybacks both acknowledgements and buffer allocations

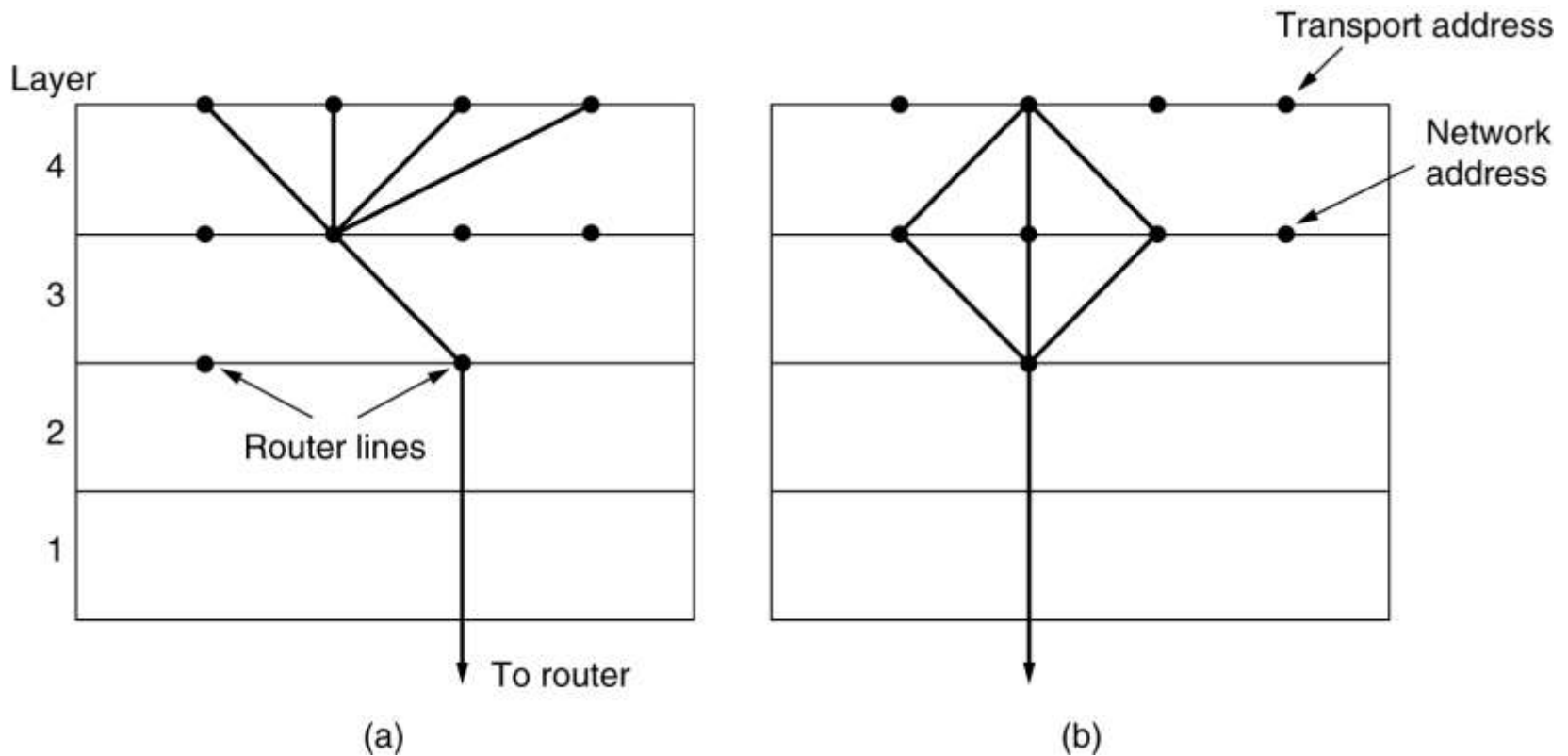
Flow Control and Buffering

Managing dynamic buffer allocation : Example

A datagram subnet with 4-bit sequence numbers

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

Multiplexing



(a) Upward multiplexing. (b) Downward multiplexing.

Crash Recovery

How to recover from host crashes?

Three events are possible at the server:

1. sending an acknowledgement (A),
2. writing to the output process (W),
3. and crashing (C).

The three events can occur in six different orderings:

AC(W), AWC, C(AW), C(WA), WAC, and WC(A)

Crash Recovery

How to recover from host crashes?

Strategy used by sending host	Strategy used by receiving host					
	First ACK, then write			First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
 DUP = Protocol generates a duplicate message
 LOST = Protocol loses a message

Different combinations of client and server strategy.